Improved-Machine-Learning-Based-Intrusion

Detection-using-Autoencoder-Ensembles

Mini ProjectReport

Team Members:
Kodem Jyothesh Kumar – 2023UAI1807

Institute:
MalaviyaNational Institute of Technology, Jaipur

Department:
AI&DataEngineering

Course:
INFORMATION SECURITY

Problem Statement

With the increasing dependence on computer networks, cyberattacks targeting local network communication have become a serious concern. One such attack is the Address Resolution Protocol (ARP) Man-in-the-Middle (MitM) attack, where an attacker intercepts communication between devices and can monitor, alter, or steal sensitive data. Detecting such attacks in real time is challenging because malicious traffic may resemble normal network behavior.

This project focuses on analyzing network traffic and identifying abnormal behavior using the Kitsune anomaly detection framework. Raw packet capture data is processed, converted into structured features, and used to detect suspicious traffic patterns that may indicate an attack.

Objectives

The main objectives of this project are:

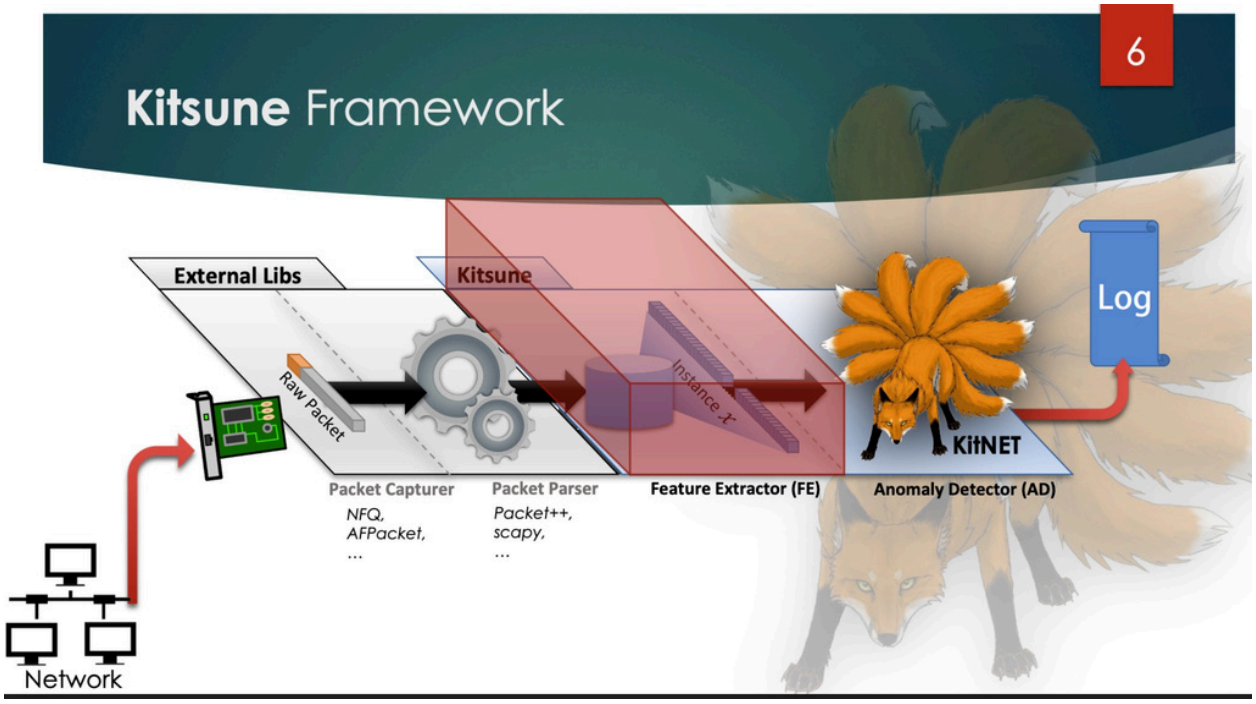
- To analyze raw network traffic captured in packet format.
- To convert packet capture data into a structured dataset containing 115 extracted features.
- To apply the Kitsune machine learning framework for anomaly detection.
- To specifically detect ARP Man-in-the-Middle (MitM) attacks within local networks.
- To achieve high detection accuracy while maintaining strong recall for both normal and malicious traffic.

- To evaluate model performance using multiple thresholds and classification metrics.
- To demonstrate how machine learning can strengthen real-time cybersecurity monitoring systems.

System Architecture

The proposed architecture utilizes a multi-stage pipeline designed to identify network anomalies by leveraging the Kitsune framework. The process begins with the conversion of raw packet capture files into structured tabular data, followed by comprehensive preprocessing, model training, anomaly scoring, and rigorous performance evaluation.

Implementation was carried out using **Google Colab**, providing a robust cloud-based infrastructure for data processing and model execution.



Workflow Description

1. The compressed packet capture archive containing ARP MitM traffic is extracted to retrieve the .pcapng file.
2. Core network traffic features are extracted from the packet capture file and converted into CSV format using **TShark**.
3. A custom preprocessing routine ensures every record adheres to the **115inputfeatures** mandated by the Kitsune model, with zero-padding applied to missing dimensions.
4. The anomaly detection model is trained exclusively on selected benign traffic samples.
5. Features are scaled between 0 and 1 through normalization using **MinMaxScaler**.

6. The Kitsune model is trained using the processed benign dataset.
7. Evaluation is performed on the complete dataset, incorporating both normal and attack traffic, using the fitted scaler.
8. Anomaly scores are generated for each sample within the evaluation set by Kitsune.
9. Traffic is classified as either normal or anomalous by comparing these scores against predefined thresholds.
10. The final performance is scrutinized using confusion matrices, score distributions, and various classification metrics.

Architecture Flow Diagram

Compressed PCAP File
↓
Extract .pcapng File
↓
TShark Feature Extraction
↓
CSV Dataset Creation
↓
115 Feature Formatting
↓
Data Normalization (MinMaxScaler)
↓
Benign Traffic for Training
↓
Kitsune Model Training
↓
Full Dataset Evaluation
↓
Anomaly Score Generation
↓
Threshold-Based Classification
↓
Metrics, Boxplots, Confusion Matrix

Key Components Used

- **TShark:** Employed for parsing packets and extracting relevant features.
- **Python / Pandas / NumPy:** Utilized for data preprocessing tasks.
- **Scikit-learn:** Applied for feature normalization.
- **Kitsune Framework:** The core engine for anomaly detection.
- **Google Colab:** Provided the environment for development and experimentation.

The project uses the **kitsune-pytorch** implementation of the Kitsune framework, an advanced machine learning system designed for real-time network anomaly detection. Kitsune is widely used in cybersecurity applications because it learns patterns of normal network traffic and detects deviations that may indicate malicious activity.

Unlike traditional supervised classification models, Kitsune follows an **unsupervised learning** approach. In this project, only benign network traffic samples were used during training. This enables the system to detect previously unseen attacks without depending on large labeled attack datasets.

Core Concepts Applied

1. Autoencoder-Based Learning

Kitsune internally uses an **ensemble of autoencoders**. Autoencoders are neural networks trained to reconstruct normal input patterns. When abnormal traffic is encountered, reconstruction becomes poor, resulting in a higher anomaly score.

2. Unsupervised Anomaly Detection

The model was trained only on normal traffic data. Instead of learning attack labels, it learns normal network behavior and flags deviations as suspicious traffic.

3. Feature Engineering

Raw packet capture traffic was transformed into structured numerical input features. A total of **115 features** were used to represent packet and traffic characteristics for model training and evaluation.

4. Data Normalization

All features were scaled using **MinMaxScaler** to a range of 0 to 1. This improves training stability and ensures balanced input values across features.

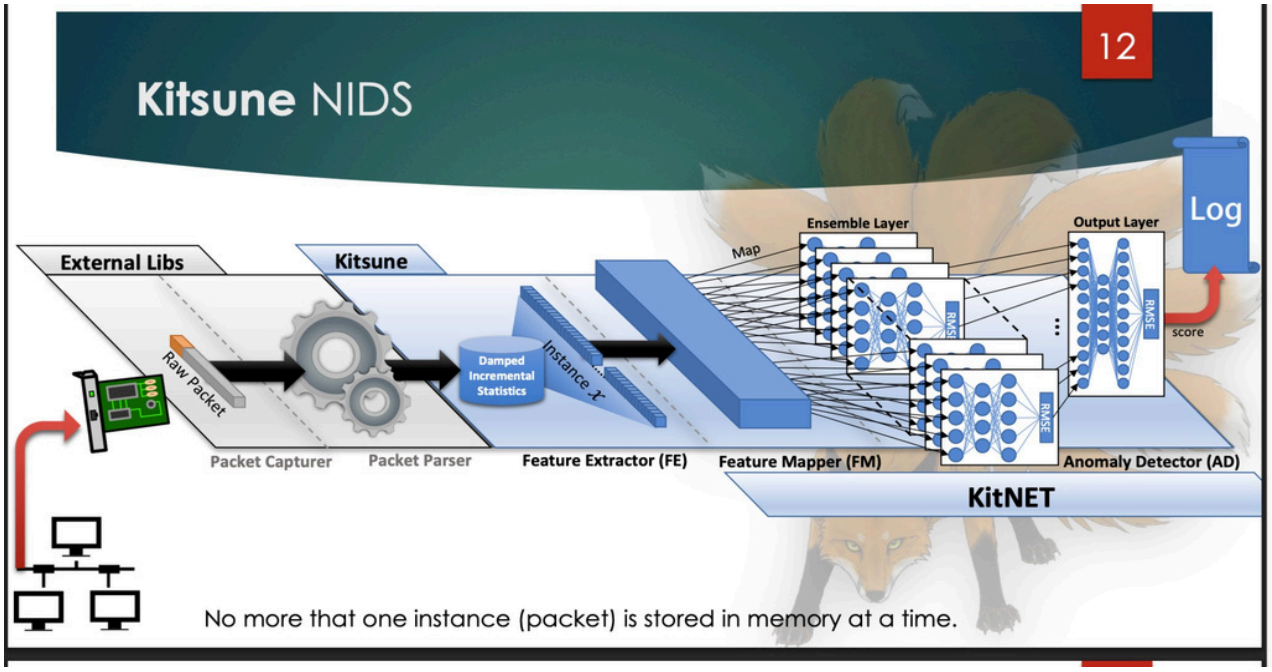
5. Reconstruction Error / Anomaly Score

The Kitsune model generates anomaly scores based on reconstruction error. Higher scores indicate stronger deviation from learned normal behavior and are more likely to represent attacks.

6. Batch Training

Model training was performed using a batch size of **32**, improving computational efficiency during learning.

Why Kitsune Was Chosen

- Effective for real-time intrusion detection
- Can detect previously unseen attacks
- Requires only benign traffic for training
- Lightweight and efficient compared to heavy classifiers
- Well suited for cybersecurity anomaly detection tasks
- 

The trained Kitsune model was evaluated on a dataset containing both normal traffic and ARP Man-in-the-Middle attack samples. The model generated anomaly scores for each network instance, which were then compared against selected threshold values to classify traffic as normal or anomalous.

Performance Overview

Initial evaluation showed that the model was effective in distinguishing benign traffic from suspicious activity. Attack samples generally received higher anomaly scores than normal traffic, demonstrating the ability of Kitsune to identify deviations from learned behavior.

Metrics Used

The following evaluation metrics were used to assess model performance:

- Accuracy
- Attack Recall

- Normal Recall
- Confusion Matrix
- Threshold-wise Performance Comparison

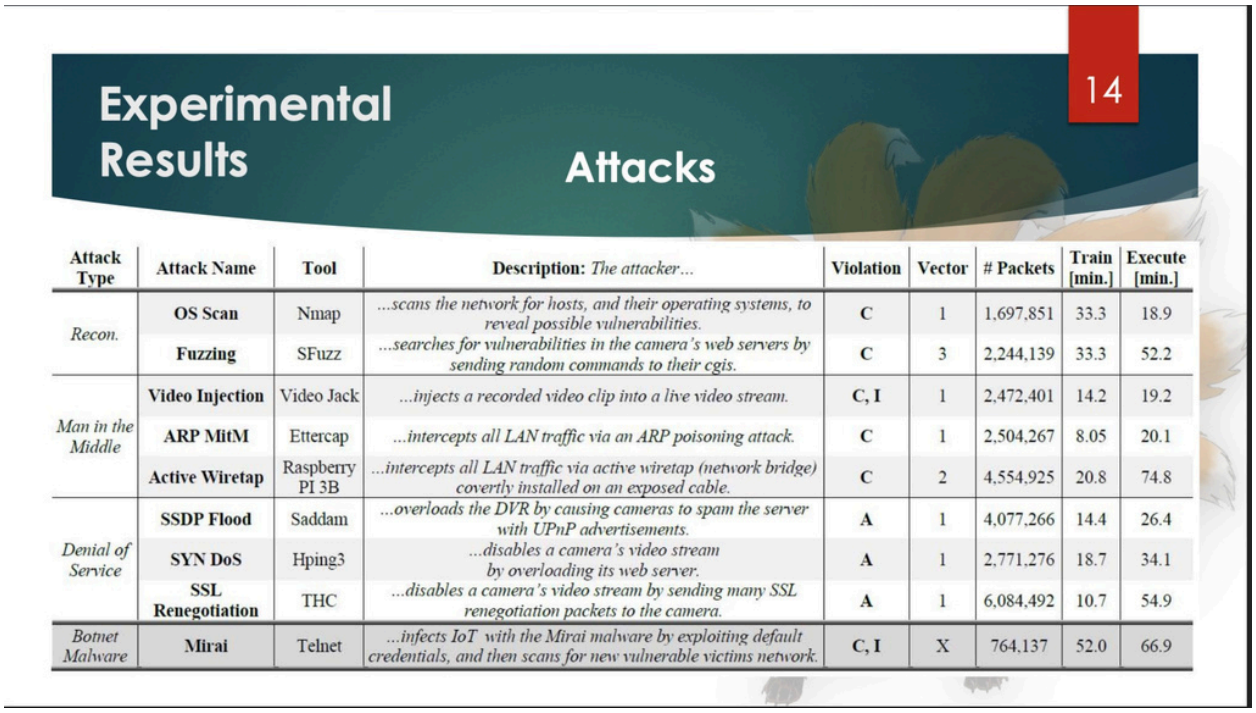
Threshold Analysis

Multiple thresholds such as **80%, 85%, 90%, 95%, 98%, and 99% quantiles** were tested to observe the trade-off between false positives and false negatives.

Lower thresholds improved attack detection sensitivity, while higher thresholds reduced false alarms. A balanced threshold provided the most stable overall performance.

Visual Analysis

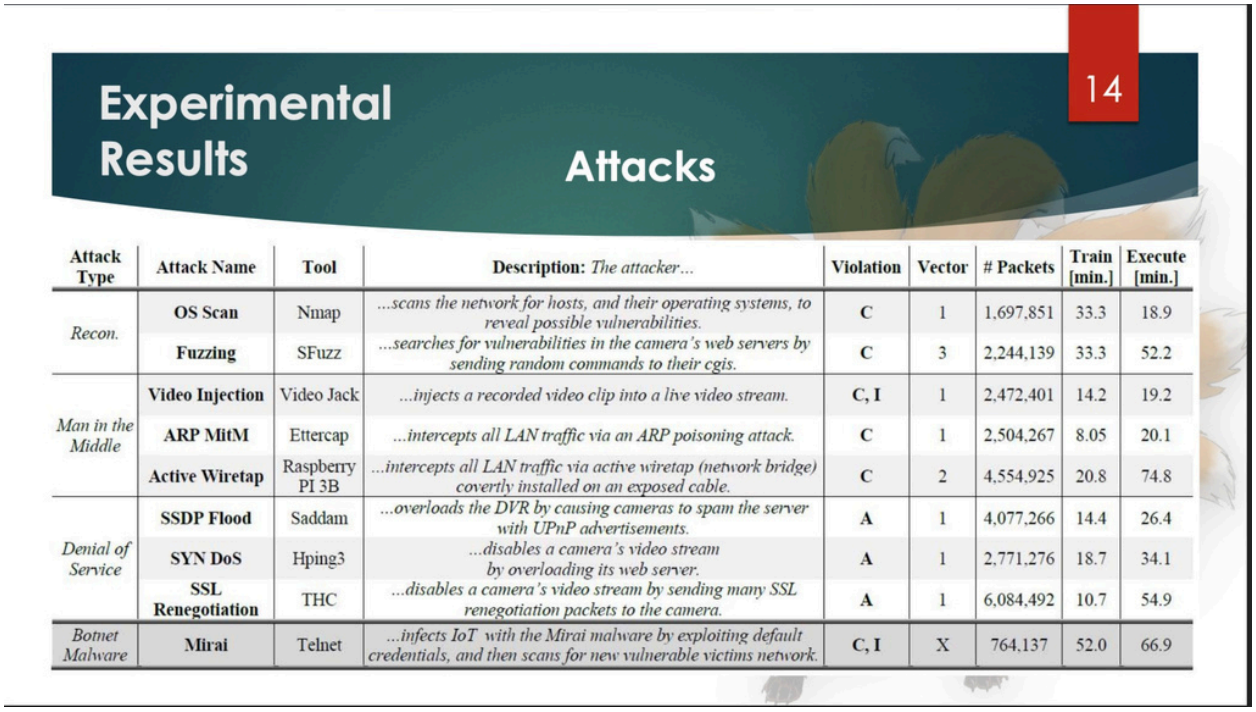
A box plot of anomaly scores showed clear separation between normal and attack traffic distributions. This indicates that anomalous samples consistently produced higher scores than benign samples.



Attack Type	Attack Name	Tool	Description: *The attacker…*	Violation	Vector	# Packets	Train [min.]	Execute [min.]
Recon.	**OS Scan**	Nmap	*…scans the network for hosts, and their operating systems, to reveal possible vulnerabilities.*	**C**	1	1,697,851	33.3	18.9
	Fuzzing	SFuzz	*…searches for vulnerabilities in the camera's web servers by sending random commands to their cgis.*	**C**	3	2,244,139	33.3	52.2
Man in the Middle	**Video Injection**	Video Jack	*…injects a recorded video clip into a live video stream.*	**C, I**	1	2,472,401	14.2	19.2
	ARP MitM	Ettercap	*…intercepts all LAN traffic via an ARP poisoning attack.*	**C**	1	2,504,267	8.05	20.1
	Active Wiretap	Raspberry PI 3B	*…intercepts all LAN traffic via active wiretap (network bridge) covertly installed on an exposed cable.*	**C**	2	4,554,925	20.8	74.8
Denial of Service	**SSDP Flood**	Saddam	*…overloads the DVR by causing cameras to spam the server with UPnP advertisements.*	**A**	1	4,077,266	14.4	26.4
	SYN DoS	Hping3	*…disables a camera's video stream by overloading its web server.*	**A**	1	2,771,276	18.7	34.1
	SSL Renegotiation	THC	*…disables a camera's video stream by sending many SSL renegotiation packets to the camera.*	**A**	1	6,084,492	10.7	54.9
Botnet Malware	**Mirai**	Telnet	*…infects IoT with the Mirai malware by exploiting default credentials, and then scans for new vulnerable victims network.*	**C, I**	X	764,137	52.0	66.9

Preliminary Conclusion

The Kitsune framework demonstrated strong potential for real-time ARP MitM attack detection. With proper threshold tuning, the system can achieve high detection rates while maintaining acceptable performance on normal traffic.



Attack Type	Attack Name	Tool	Description: *The attacker…*	Violation	Vector	# Packets	Train [min.]	Execute [min.]
Recon.	**OS Scan**	Nmap	*…scans the network for hosts, and their operating systems, to reveal possible vulnerabilities.*	**C**	1	1,697,851	33.3	18.9
	Fuzzing	SFuzz	*…searches for vulnerabilities in the camera's web servers by sending random commands to their cgis.*	**C**	3	2,244,139	33.3	52.2
Man in the Middle	**Video Injection**	Video Jack	*…injects a recorded video clip into a live video stream.*	**C, I**	1	2,472,401	14.2	19.2
	ARP MitM	Ettercap	*…intercepts all LAN traffic via an ARP poisoning attack.*	**C**	1	2,504,267	8.05	20.1
	Active Wiretap	Raspberry PI 3B	*…intercepts all LAN traffic via active wiretap (network bridge) covertly installed on an exposed cable.*	**C**	2	4,554,925	20.8	74.8
Denial of Service	**SSDP Flood**	Saddam	*…overloads the DVR by causing cameras to spam the server with UPnP advertisements.*	**A**	1	4,077,266	14.4	26.4
	SYN DoS	Hping3	*…disables a camera's video stream by overloading its web server.*	**A**	1	2,771,276	18.7	34.1
	SSL Renegotiation	THC	*…disables a camera's video stream by sending many SSL renegotiation packets to the camera.*	**A**	1	6,084,492	10.7	54.9
Botnet Malware	**Mirai**	Telnet	*…infects IoT with the Mirai malware by exploiting default credentials, and then scans for new vulnerable victims network.*	**C, I**	X	764,137	52.0	66.9

Challenges Faced and How We Solved

References

Thefollowingresourceswere referred to during the development and execution of this project:

1. Guillem96. **kitsune-pytorch** GitHub Repository. Available at: https://github.com/Guillem96/kitsune-pytorch
2. Kaggle. **Network Traffic / Cybersecurity Dataset** used for ARP MitM traffic analysis and packet capture samples.
3. Mirsky, Y., Doitshman, T., Elovici, Y., and Shabtai, A. **Kitsune: An Ensemble of Autoencoders for Online Network Intrusion Detection.** Network and Distributed System Security Symposium (NDSS), 2018.
4. Python Software Foundation. **Python Programming Language Documentation.**
5. Pedregosa et al. **Scikit-learn: Machine Learning in Python.** Used for preprocessing and evaluation metrics.
6. McKinney, W. **Pandas Library Documentation.** Used for dataset handling and analysis.
7. Wireshark Foundation. **TShark Command-Line Packet Analyzer Documentation.** Used for packet capture feature extraction.
8. OpenAI. **ChatGPT** – Used for debugging assistance, technical explanations, and documentation support.
9. Google. **Gemini** – Used for ideation, explanations, and project assistance.

AI Tools Used

- ChatGPT
- Google Gemini

Areas of Usage

The above AI tools were used during multiple stages of the project, including:

- Coding assistance and script improvement
- Debugging errors and fixing dependencies
- Understanding the Kitsune framework and project workflow
- Resolving Google Colab environment issues
- File path and dataset handling support
- Data preprocessing suggestions
- Result interpretation and evaluation understanding
- Report writing and documentation preparation
- README content generation

Scope of Usage

LLM tools were used in all major areas of project development **except architecture/diagram generation**.

Prompt Record Maintenance

All prompts used during the project have been collected and stored in the **LLM_Prompts/** folder as required in the submission instructions.

Final Responsibility

Although AI tools were used as assistants, all final implementation decisions, testing, modifications, and verification were carried out by the project team.